

Test Strategy

How QA tests the Workbench, mapped onto the architecture the team already knows. What each layer proves, how much confidence it gives, and how we know a passing test actually means something.

OWNER	VERSION	AUDIENCE	OUT OF SCOPE
Shanthi Subramanian · BA / QA	1.0 · 21 September 2026	Whole project team	Unit testing — Dev owned

01 Scope & ownership

Who tests what, so nothing falls between us and nothing is done twice.

LAYER	OWNER	ANSWERS
Unit	DEV	Is this class's logic correct in isolation?
API	QA	Does the service behave correctly over HTTP, for the right people, in the right order?
Data	QA	Does the database itself refuse to store something invalid?
UI	QA	Can a real person complete their job end to end?
AI evaluation	QA	Is the model's output good enough to put in front of an analyst?

Unit testing is out of QA's scope. It belongs with the people who wrote the code and refactor it — it is their safety net, and splitting ownership of it would slow everyone down. Dev currently has 62 unit test cases, all passing, and QA treats them as a given rather than re-proving the same ground.

THE RULE THAT PREVENTS DUPLICATED EFFORT

Every check happens at the cheapest layer that can actually see it. API tests never re-assert scoring arithmetic — unit tests do that in milliseconds. UI tests never check a status code — a browser is the most expensive possible way to read one. Each layer only covers what the layers beneath it structurally cannot reach.

02 Architecture & test points

Where each kind of test plugs into the system we actually built.

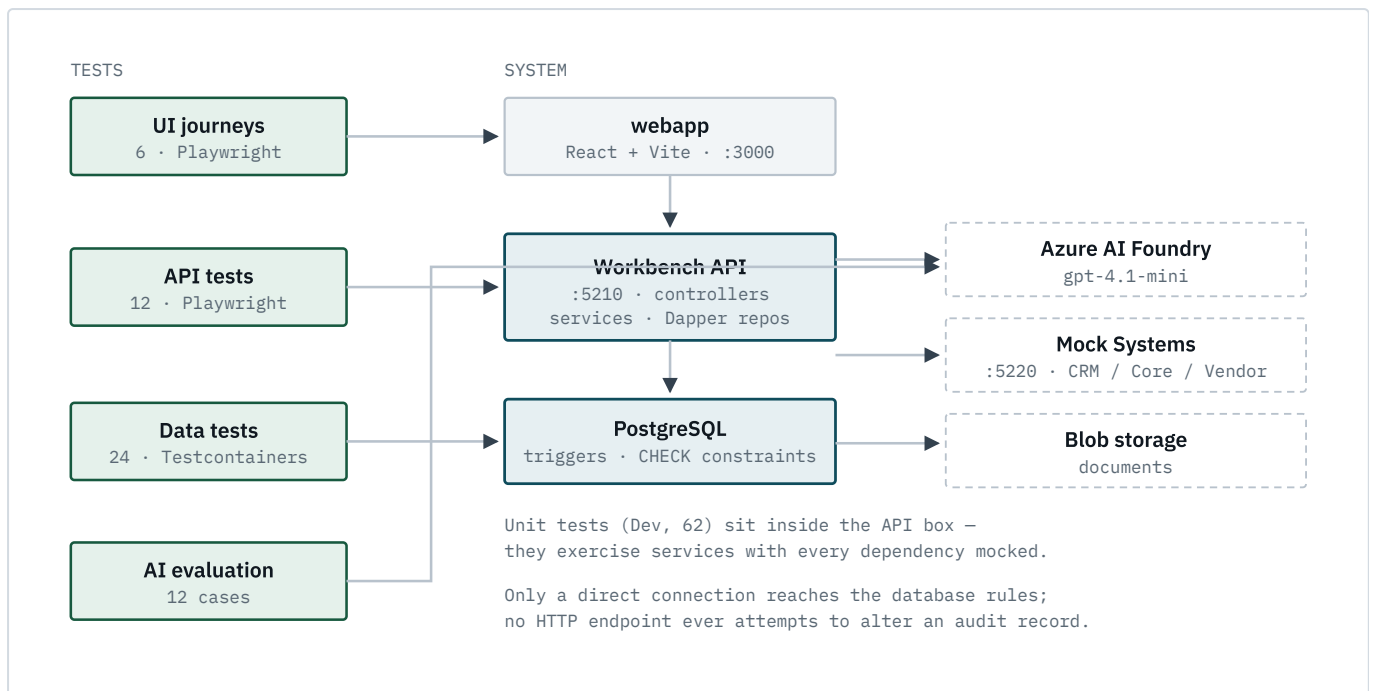


Figure 1. Four QA test layers, each entering the system at a different point. The layer that matters most for governance is the lowest one: the append-only audit trigger and the risk-score constraints live in PostgreSQL and cannot be reached through the API at all.

The four test layers

What each one proves, and why it exists separately.

API tests PLAYWRIGHT • 12 TODAY

The workhorse. No browser, runs in seconds, and it is where authorization actually lives. Four questions per write endpoint:

1. **Happy path** — correct input produces the correct result
2. **Bad input** — a blank override reason, an out-of-range value, an unsupported file type is refused
3. **Wrong person** — a Product Owner cannot cast a committee vote
4. **Wrong time** — a vote before routing, a finalize before review, is refused

Data tests TESTCONTAINERS • 24 PLANNED

Connects directly to a throwaway PostgreSQL container with the real schema. This is the **only** layer that can prove the governance rules the product is judged on, because they are enforced by the database rather than by application code:

- The audit trail is genuinely append-only — `UPDATE` and `DELETE` are rejected by a trigger
- Residual risk can never be stored as zero
- A mitigation factor of 1.0 is rejected
- An override without a reason is rejected
- Every state change writes its audit entry *in the same transaction* — no silent gaps

UI journeys PLAYWRIGHT • 6 TODAY

Three role journeys — Product Owner raises, Analyst assesses and finalizes, Committee votes — run **serially as one story**: the same request moving through the workflow. This single path is the product demo and exercises the whole governance chain.

Deliberately small. UI tests need four processes running and are the slowest thing we own, so they answer only the question no cheaper layer can: *can a real person complete their job?* Twelve is a ceiling, not a target.

AI evaluation 12 CASES • 2 EXECUTING

Scores the three AI touchpoints — category mapping, document extraction, narrative drafting — against fixed datasets. Deterministic wherever the domain allows: category mapping is scored by exact set-overlap against the FFIEC codes, because the category list is small and fixed.

Narrative quality is scored by a **labelled heuristic** — citation presence and well-formedness — and the eval documentation says so plainly. A narrative eval that quietly claimed more than it checks would be worse than none.

Coverage by component

Honest confidence ratings, component by component. This is the table to scan.

COMPONENT	WHAT COULD GO WRONG	COVERED BY	CONFIDENCE
Risk scoring Epic 7	The number is wrong; controls zero out risk	Unit (Dev) + API + Data	 HIGH
Committee workflow Epic 8	Wrong resolution; a vote lost or anonymised	Unit (Dev) + API + UI	 HIGH
Intake Epic 1	Bad data accepted; request lost	Unit (Dev) + API + UI	 HIGH
Finalization gate Epic 6	An unreviewed assessment reaches committee	Unit (Dev) + UI	 MEDIUM
AI touchpoints Epics 2, 4, 5	Fabricated citation; silent bad output	Unit (Dev) + evals	 MEDIUM
Data ingestion Mock Systems	Intake fails when a downstream system is down	Unit (Dev)	 MEDIUM
Access control cross-cutting	Anyone can act as anyone	API	 LOW
Audit trail Epic 9	A record is altered after the fact	Data tests — <i>not yet built</i>	 LOW
Policy research Epic 3	Wrong or no policy surfaced	— nothing yet	 LOW
Configuration Epic 10	A config change breaks in-flight assessments	Unit (Dev) + API	 MEDIUM

 **High** — covered at more than one layer, including one that cannot be bypassed. ·  **Medium** — covered, but at one layer only, or with a known gap. ·  **Low** — little or no automated coverage today.

WHERE THE TEAM CAN BE CONFIDENT RIGHT NOW

Risk scoring, the committee workflow and intake are solid. Each is covered at two or three independent layers, and the scoring rule that matters most — residual risk can never reach zero — is guaranteed by arithmetic and enforced in three separate places: the formula's own bounds, a C# guard, and a database constraint. That one is not a matter of remembering to check.

WHERE WE ARE NOT YET CONFIDENT, AND WHY

The audit trail and access control are the two real gaps. Both are fixable and both are scheduled. Audit immutability needs the Data layer, which is specified but not yet built — it is the highest-value work remaining. Access control has tests written; they currently record a known gap rather than passing, which is deliberate and explained in section 5.

Policy research has no coverage because it is deterministic full-text search rather than AI, and has been the lowest risk of the ten epics. That is a conscious ordering decision, not an oversight.

Why a green board means something

The most reasonable question to ask QA. Three practices answer it.

1 · We break the code on purpose to check the tests notice

A passing test proves nothing until you have seen it fail for the right reason. Before trusting any governance test, we invert the rule it guards and confirm precisely that test goes red — then revert.

Done for the finalization gate: disabling it failed **exactly five tests** and left the other seven green, because those exercise a different path. That selectivity is the signal. A suite that failed more, or fewer, would be telling us something we did not intend.

2 · Known gaps are recorded as tests, never deleted

Where the product does not yet do the right thing, the test asserts the **correct** behaviour and is marked expected-to-fail. Three things follow:

- The board stays green while the gap is open, so it stays readable
- The moment someone fixes it, the runner reports *"expected to fail, but passed"* — **the fix announces itself**
- The requirement is never lost, which is what happens if you delete the test or soften it to match the bug

This is why some suites show expected failures. They are not broken tests; they are open items with a tripwire attached.

3 · A dead environment can never look like a pass

An expected-to-fail test passes when it fails for *any* reason — including the server being down. So every suite first confirms its dependency is reachable and stops loudly if not. Without that, a suite run against nothing would produce a fully green board proving absolutely nothing.

AND EVERY TEST NAMES THE STORY IT PROVES

```
test('US-8.2 AC5 — a vote is recorded against the member by name', ...)
```

Both test runners write these names into their results, so traceability back to the Azure DevOps board is produced from a build rather than maintained by hand. A story with no matching test shows up as an empty row on its own.

Manual testing

Six scenarios stay manual — not because automation is hard, but because it would answer the wrong question.

SCENARIO	WHY A HUMAN
AI narrative quality	The eval checks a narrative is well-formed and cites something. It cannot judge whether the citation is <i>apposite</i> or the reasoning sound. That is domain judgement
Examiner-readiness of the audit export	The requirement is a format "suitable for handing to an examiner". No assertion evaluates that; only reading it does
Exploratory testing of overrides	Override reasons are free text. Scripted tests only find what they were told to look for
First deployment walkthrough	Smoke checks confirm services answer. Only a person catches a wrong-but-present environment variable
Accessibility spot-check	Scanners catch roughly a third of real problems. The Assessment Workspace is where an analyst lives all day
Committee decision semantics	The quorum rule is proven. Whether the workflow makes sense to people <i>deliberating</i> is a human question

Each gets a short written outcome recorded — date, who ran it, what they found. A manual pass with no note is indistinguishable from a manual test that never ran.

07

When tests run

Fast feedback on every change; the expensive things later.

STAGE	RUNS	BLOCKS THE MERGE?	TIME
Build	Every PR	YES	~2 min
Unit tests (Dev)	Every PR	YES	~1 min
Data tests	Every PR	YES	~3 min
API tests	Every PR	YES	~3 min
UI journeys	On merge	SOFT AT FIRST	~5 min
AI evaluation	Nightly	REPORT ONLY	~3 min

PR feedback stays under ten minutes. That is a design constraint, not an aspiration — a gate slower than that gets bypassed, and a bypassed gate is worse than no gate because it creates false confidence.

UI journeys run on merge rather than per-PR because they need four processes and a browser. Once they have been stable for a week they move to a hard gate.

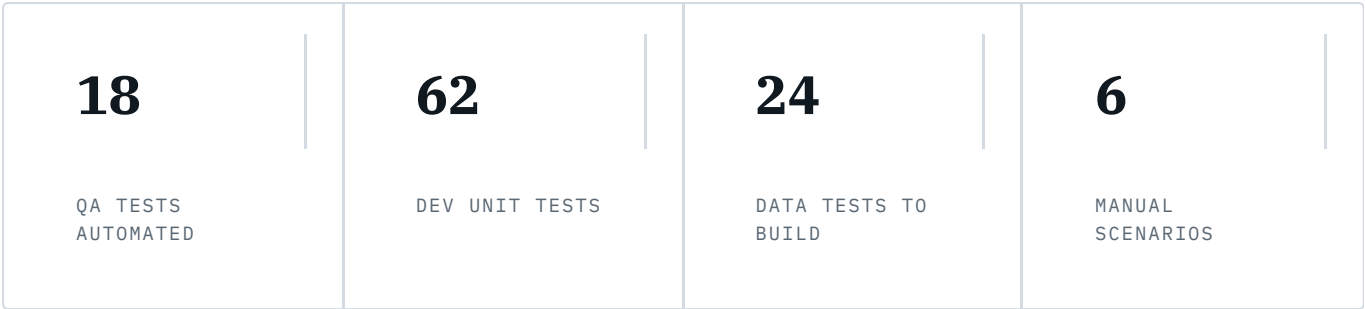
CURRENT STATE, STATED PLAINLY

The application PR pipeline exists and gates **build and unit tests** today. The API, Data and UI stages are **not yet wired in** — the tests exist and pass locally, but they do not run automatically. Closing that is additive: a Postgres service container and three steps. It is the top item on the plan in section 8.

08

Where we are

Current position and the next five things, in order.



#	NEXT	GIVES US	OWNER
1	Wire API tests into the PR pipeline	12 existing tests start protecting every change	DevOps + QA
2	Build the Data test suite	Audit immutability and the scoring constraints move from <i>Low</i> to <i>High</i> confidence	QA
3	Run the AI evaluations against the live model	10 blocked cases start producing real results	QA + Dev
4	Add UI journeys to the merge pipeline	The golden path is protected before any demo	DevOps + QA
5	Execute the six manual scenarios	The judgement-based checks are done and recorded	QA

Items 1 and 3 need something from outside QA — a pipeline change and a configured model endpoint. Both are small, and both are on the critical path for the coverage ratings in section 4 to improve.

HOW TO READ THIS STRATEGY IN ONE SENTENCE

We test each thing once, at the cheapest layer that can see it; we prove the tests work by breaking the code on purpose; and where we are not yet confident, this document says so rather than rounding up.

Defects

Open findings are tracked separately in the QA defect register and on the Azure DevOps board, each with a reproduction and an owner. They are deliberately not listed here — this document is about how we test, not what is currently broken.

Allowed values, numeric bounds and component boundaries in this document were read from the codebase and the database schema rather than from documentation, so they reflect what the system actually enforces. Test counts are as of 21 September 2026. Companion documents: *Test Starter Guide* (how to begin), *QA Test Case Register* (every case), *Test Strategy Alignment Review* (pipeline status).